Antmicro

Topwrap

2024-12-03

DOCUMENTATION

1 Introduction to Topwrap **1**

2 Installing Topwrap **2**
2.1 1. Install required system packages . . . 2
2.2 2. Install the Topwrap user package . . . 2
2.3 3. Verify the installation . . . 3

3 Getting started **4**
3.1 Design overview . . . 4
3.2 Parsing Verilog files . . . 4
3.3 Building designs with Topwrap . . . 5
3.4 Command-line flow . . . 8

4 Advanced options **11**
4.1 Creating block designs in the GUI . . . 11
4.2 Command Line Interface (CLI) . . . 14

5 Sample projects **15**
5.1 Embedded GUI . . . 15
5.2 Constant . . . 15
5.3 Inout . . . 16
5.4 Hierarchy . . . 18
5.5 PWM . . . 18
5.6 HDMI . . . 19
5.7 SoC . . . 20

6 Creating a design **22**
6.1 Design description . . . 22
6.2 IP description files . . . 25
6.3 Interface description files . . . 28

7 Configuration **30**
7.1 Configuration file location . . . 30
7.2 Configuration precedence . . . 30
7.3 Available config options . . . 31

8 Constructing, configuring and loading repositories **32**
8.1 Using the open source IP cores library with Topwrap . . . 32

9 Interconnect generation **34**

9.1 Format . . . 36
9.2 Supported interconnect types . . . 37

10 Using FuseSoC for automation **38**
10.1 Default tool for synthesis, bitstream generation and programming the FPGA . . . 38
10.2 Additional build options . . . 38
10.3 `.core` file template . . . 39
10.4 Synthesis . . . 39

11 Setup **40**

12 Code style **41**
12.1 Lint with nox . . . 41
12.2 Lint with pre-commit . . . 41
12.3 Tools . . . 42

13 Tests **43**
13.1 Test execution . . . 43
13.2 Test coverage . . . 44
13.3 Updating kpm test data . . . 44

14 Wrapper **45**

15 IPWrapper class **46**

16 IPConnect class **48**

17 ElaboratableWrapper class **53**

18 Wrapper Port **54**

19 FuseSocBuilder **56**

20 Interface Definition **58**

21 Config **59**

22 Deducing interfaces **60**
22.1 Step 1 - splitting ports into subsets . . . 60
22.2 Step 2 - matching ports with interface signal names . . . 61
22.3 Step 3 - inferring interface direction . . . 61
22.4 Step 4 - computing interface matching score . . . 61

23 Examples **63**

24 Future planned enhancements in Topwrap **64**
24.1 Library of open-source cores . . . 64
24.2 Support for hierarchical block designs in Topwrap's GUI . . . 64
24.3 Support for parsing SystemVerilog sources . . . 64

25 Other possible improvements **65**
25.1 Ability to produce top-level wrappers in VHDL . . . 65
25.2 Bus management . . . 65
25.3 Improve the process of recreating a design from a YAML file . . . 65

25.4 Deeper integration with other tools 66
25.5 Provide a way to parse HDL sources from the GUI level 66

26 Using KPM iframes inside docs **67**
26.1 Usage 67
26.2 Tests 67

Index **69**

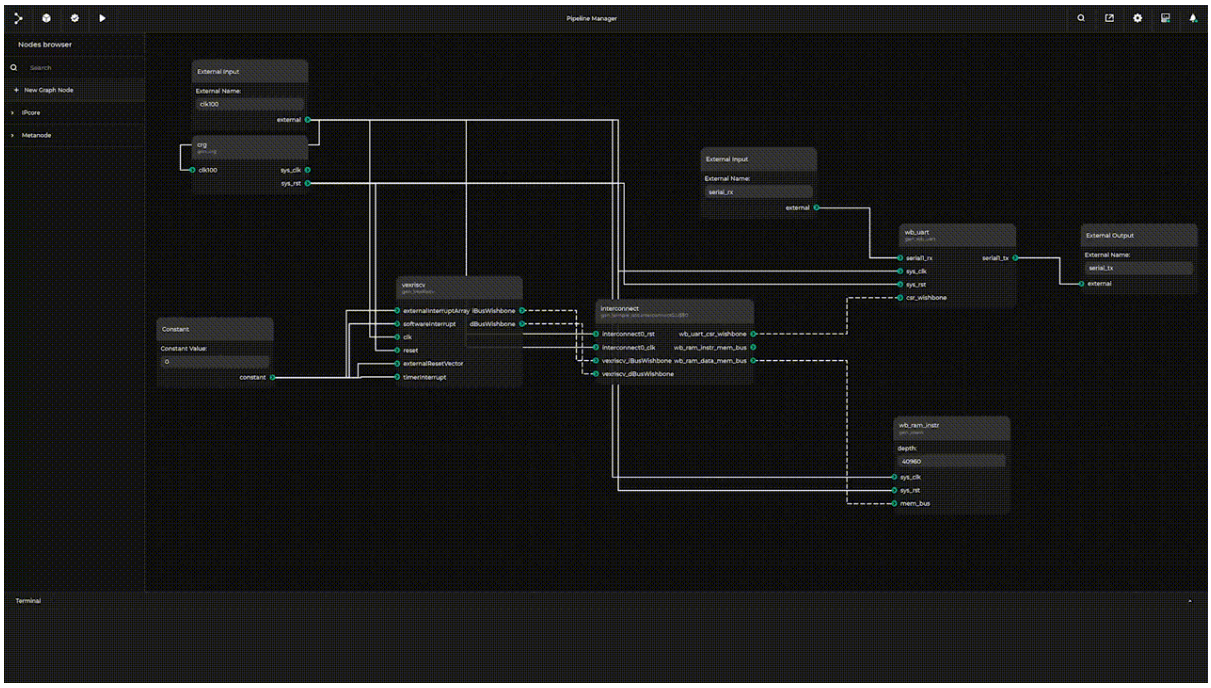
CHAPTER

ONE

INTRODUCTION TO TOPWRAP

Topwrap leverages modularity to enable the reuse of design blocks across different projects, facilitating the transition to automated logic design. It provides a standardized approach for organizing blocks into various configurations, making top-level designs easier to parse and process automatically.

As a tool, Topwrap makes it *straightforward to build* complex and *synthesizable designs* by generating a design file. The combination of *GUI and CLI-based* configuration options provides for fine-tuning possibilities. Packaging multiple files is accomplished by including them in a *custom user repository*, and an internal API enables repository creation using Python.



CHAPTER

TWO

INSTALLING TOPWRAP

2.1 1. Install required system packages

> **Warning:** The script below requires root privileges as it directly interfaces with your filesystem and package manager.
>
> Running scripts and executables as root without first verifying their contents can pose significant security risks. Always ensure their integrity and source before execution.

```
curl -fO https://raw.githubusercontent.com/antmicro/topwrap/refs/heads/main/
↪install-deps.sh
chmod +x ./install-deps.sh
sudo ./install-deps.sh
```

2.2 2. Install the Topwrap user package

Recommended: Use pipx to directly install Topwrap as a user package:

```
pipx install "topwrap[parse]@git+https://github.com/antmicro/topwrap"
```

If you can't use pipx, you can use regular pip instead. It may be necessary to do it in a Python virtual environment, such as venv:

```
python3 -m venv venv
source venv/bin/activate
pip install "topwrap[parse]@git+https://github.com/antmicro/topwrap"
```

2.3 3. Verify the installation

Make sure that Topwrap was installed correctly and is available in your shell:

```
topwrap --help
```

This should print out the help string with Topwrap subcommands listed out.

See also:

If you want to contribute to the project, check the *Developer's setup guide* for more information.

CHAPTER

THREE

GETTING STARTED

The purpose of this chapter is to provide a step by step guide on how to create a simple design with Topwrap. All the necessary files needed to follow this guide are in the examples/getting_started_demo directory.

Important

If you haven't installed Topwrap yet, go to the *Installation chapter* and make sure to install the additional dependencies for `topwrap parse`.

3.1 Design overview

We are going to create a design that will be visually represented in an interactive GUI, as seen below.

Note: An interactive KPM frame, where you can explore the block design for this section, is available here in the HTML version of this documentation.

It consists of two cores: `simple_core_1` and `simple_core_2` that connect to each other and to an external input/output.

Note: Metanodes are always utilized in designs to represent external input/output ports, module hierarchy ports or constant values. They can be found in the "Metanode" section.

3.2 Parsing Verilog files

The first step when creating designs is to parse Verilog files into IP core description YAMLs that are understood by Topwrap.

The `verilogs` directory contains two Verilog files, `simple_core_1.v` and `simple_core_2.v`.

To generate the IP core descriptions from these Verilog files run:

```
topwrap parse verilogs/{simple_core_1.v,simple_core_2.v}
```

Topwrap will generate two files `simple_core_1.yaml` and `simple_core_2.yaml` that represent the corresponding Verilog files.

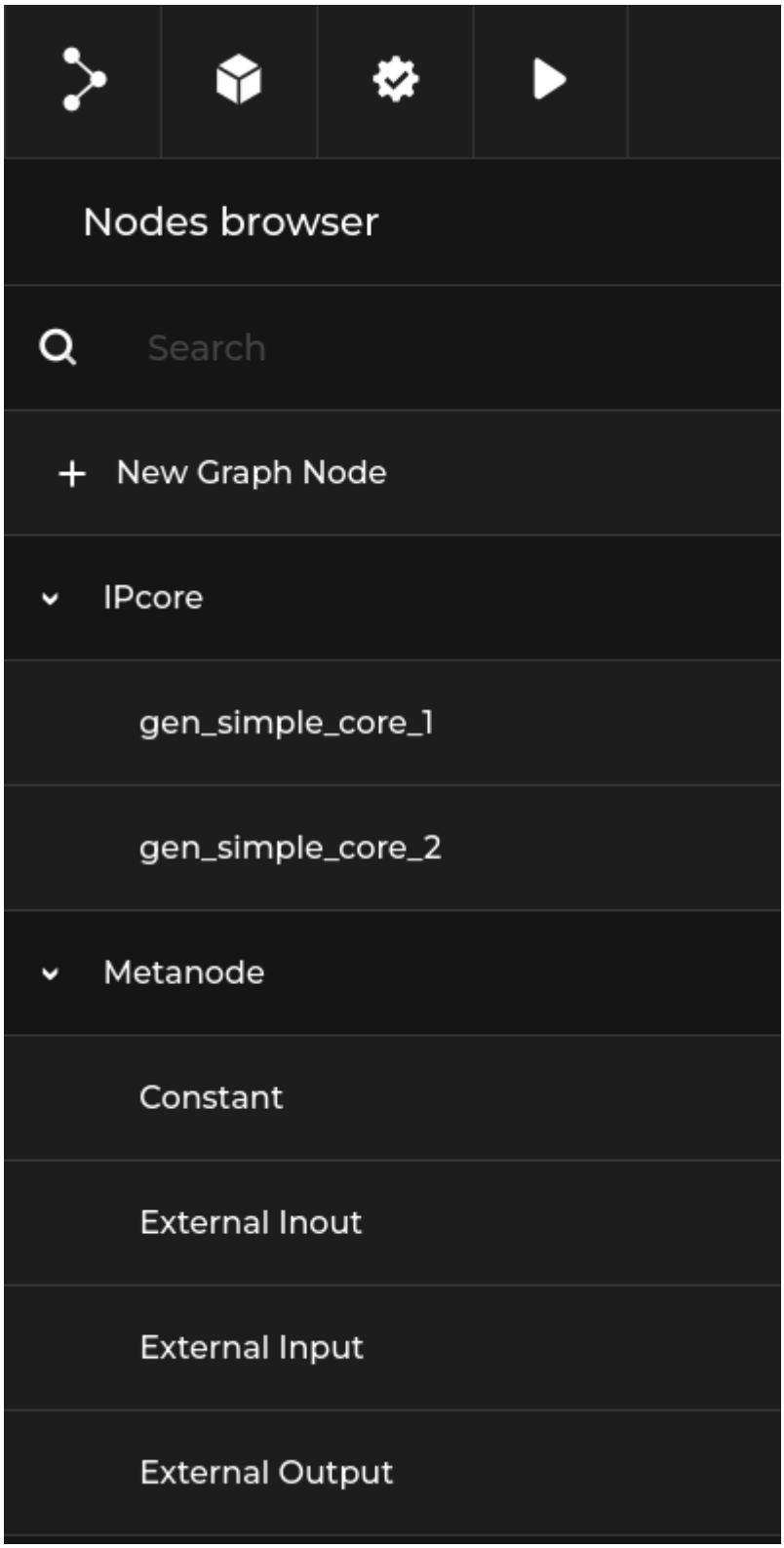
3.3 Building designs with Topwrap

3.3.1 Creating the design

The generated IP core YAMLs can be loaded into the GUI, using:

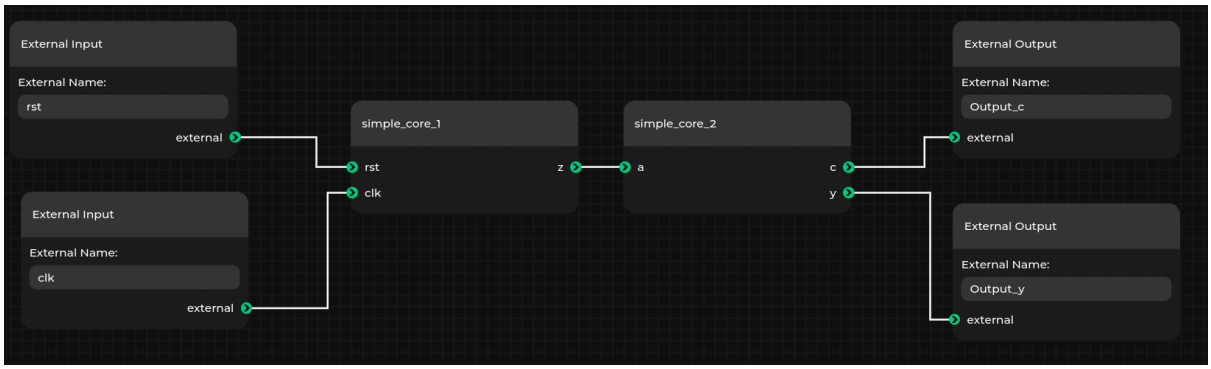
```
topwrap gui simple_core_1.yaml simple_core_2.yaml
```

The loaded IP cores can be found in the IPcore section:



With these IP cores and default metanodes, you can easily create designs by dragging and connecting cores.

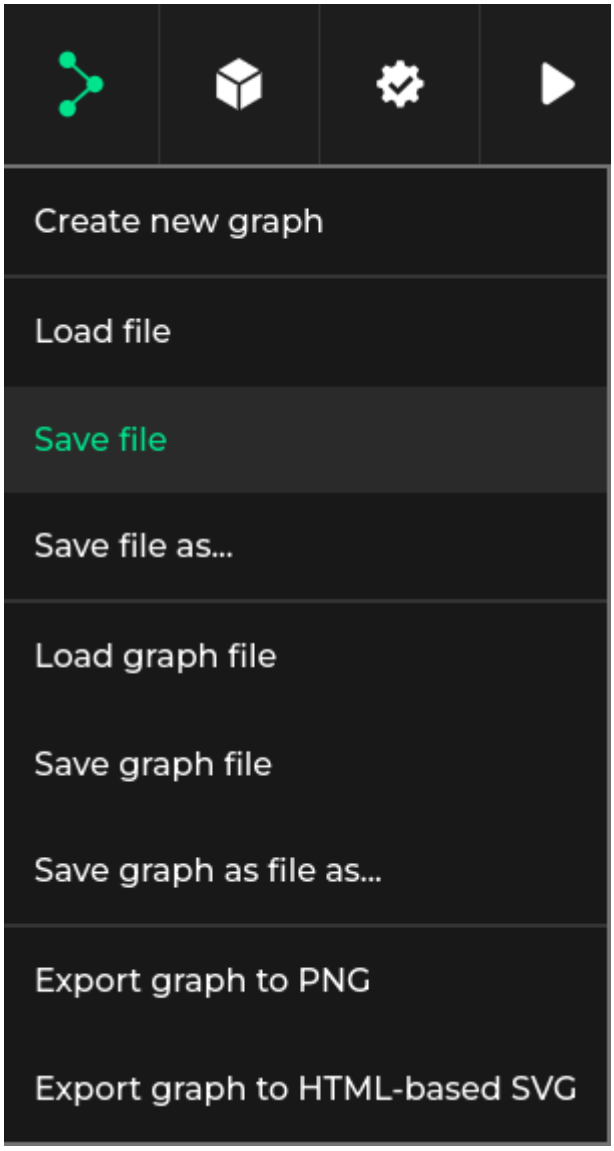
Let's make the design from the demo in the *introduction*.



Note: You can change the name of an individual node by right clicking on it and selecting `rename`. This is useful when creating multiple instances of the same IP core.

You can save the project in the *Design Description* format, which is used by Topwrap to represent the created design.

To do this, select the graph button and select `Save file`.



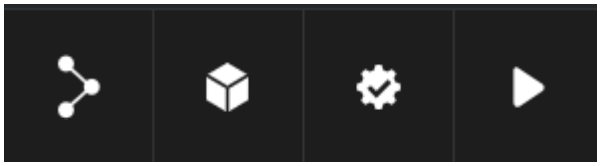
Note: The difference between `Save file` and `Save graph file` lies in which format is used for saving.

`Save file` will save the design description in the YAML format which Topwrap uses.

`Save graph file` will save the design in the graph JSON format which the GUI uses. You should only choose this one if you have a specific custom layout of the nodes in the design and you want to save it.

3.3.2 Generating Verilog in the GUI

You can generate Verilog from the design created in the previous section if you have the example running as described in the previous section. On the top bar, these four buttons are visible:



1. Save/Load designs.
2. Toggle the node browser.
3. Validate the design.
4. Build the design. If it does not contain errors, a top module will be created in the directory where `topwrap gui` was run.

3.4 Command-line flow

3.4.1 Creating designs

The manual creation of designs requires familiarity with the *Design Description* format.

First, include all the IP core files needed in the `ips` section.

```
ips:
  simple_core_1:
    file: simple_core_1.yaml
  simple_core_2:
    file: simple_core_2.yaml
```

Here, the name of the node is declared, and the IP core `simple_core_1.yaml` is named `simple_core_1` in the GUI.

Now we can start creating the design under the `design` section. The design doesn't have any parameters set, so we can skip this part and go straight into the `ports` section. In there, the connections between IP cores are defined. In the demo example, there is only one connection - between `simple_core_1` and `simple_core_2`.

In our design, it is represented like this:

```yaml
design:
  ports:
    simple_core_2:
      a:
      - simple_core_1
      - z
```

Notice that `input` is connected to `output`. All that is left to do is to declare the external connections to metanodes, like this:

```yaml
external:
  ports:
    in:
    - rst
    - clk
    out:
    - Output_y
    - Output_c
```

Now connect them to IP cores.

```yaml
design:
  ports:
    simple_core_1:
      clk: clk
      rst: rst
    simple_core_2:
      a:
      - simple_core_1
      - z
      c: Output_c
      y: Output_y
```

The final design:

```yaml
ips:
  simple_core_1:
    file: simple_core_1.yaml
  simple_core_2:
    file: simple_core_2.yaml
design:
  ports:
    simple_core_1:
      clk: clk
      rst: rst
    simple_core_2:
      a:
      - simple_core_1
      - z
      c: Output_c
      y: Output_y
```

(continues on next page)

(continued from previous page)

```
external:
  ports:
    in:
    - rst
    - clk
    out:
    - Output_y
    - Output_c
```

3.4.2 Generating Verilog top files

Info

Topwrap uses Amaranth for generating Verilog top files.

To generate the top file, use `topwrap build` and provide the design. To do this, ensure you are in the `examples/getting_started_demo` directory and run:

```
topwrap build --design {design_name.yaml}
```

Where `{design_name.yaml}` is the design saved at the end of the previous section. This will generate a `top.v` Verilog top wrapper in the specified build directory (`./build` by default).

3.4.3 Synthesis & FuseSoC

You can additionally generate a *FuseSoC core* file during `topwrap build` to automate further synthesis and implementation by simply adding the `-f` (`--fuse`) option.

CHAPTER

FOUR

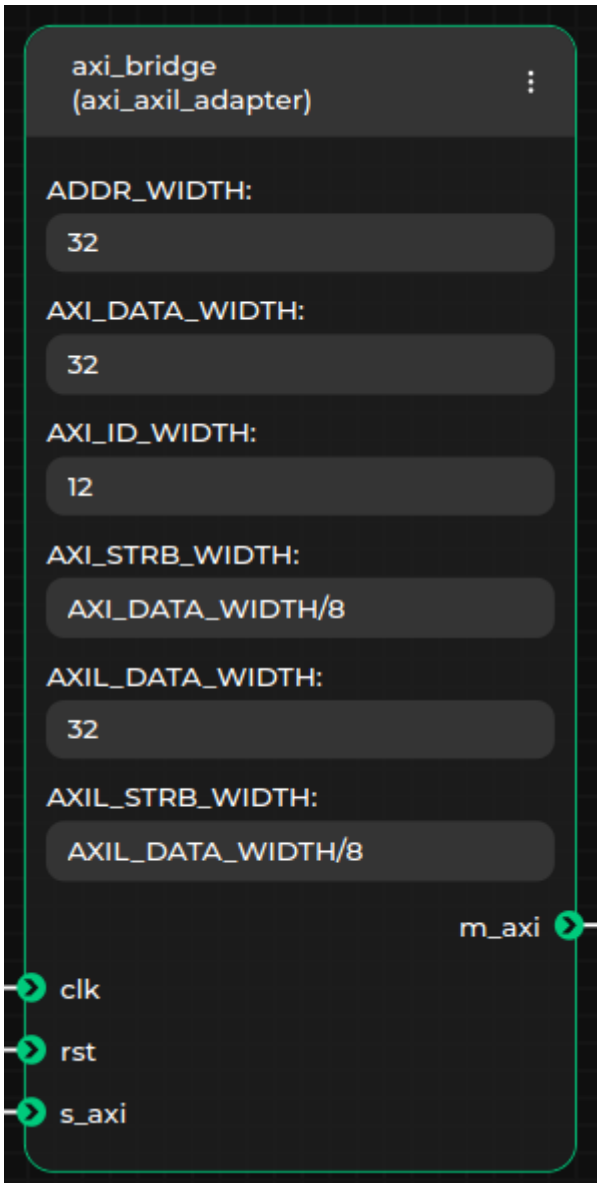
ADVANCED OPTIONS

This chapter builds upon the content covered in the *Getting started* chapter. If you have not yet reviewed it, we recommend doing so before proceeding.

4.1 Creating block designs in the GUI

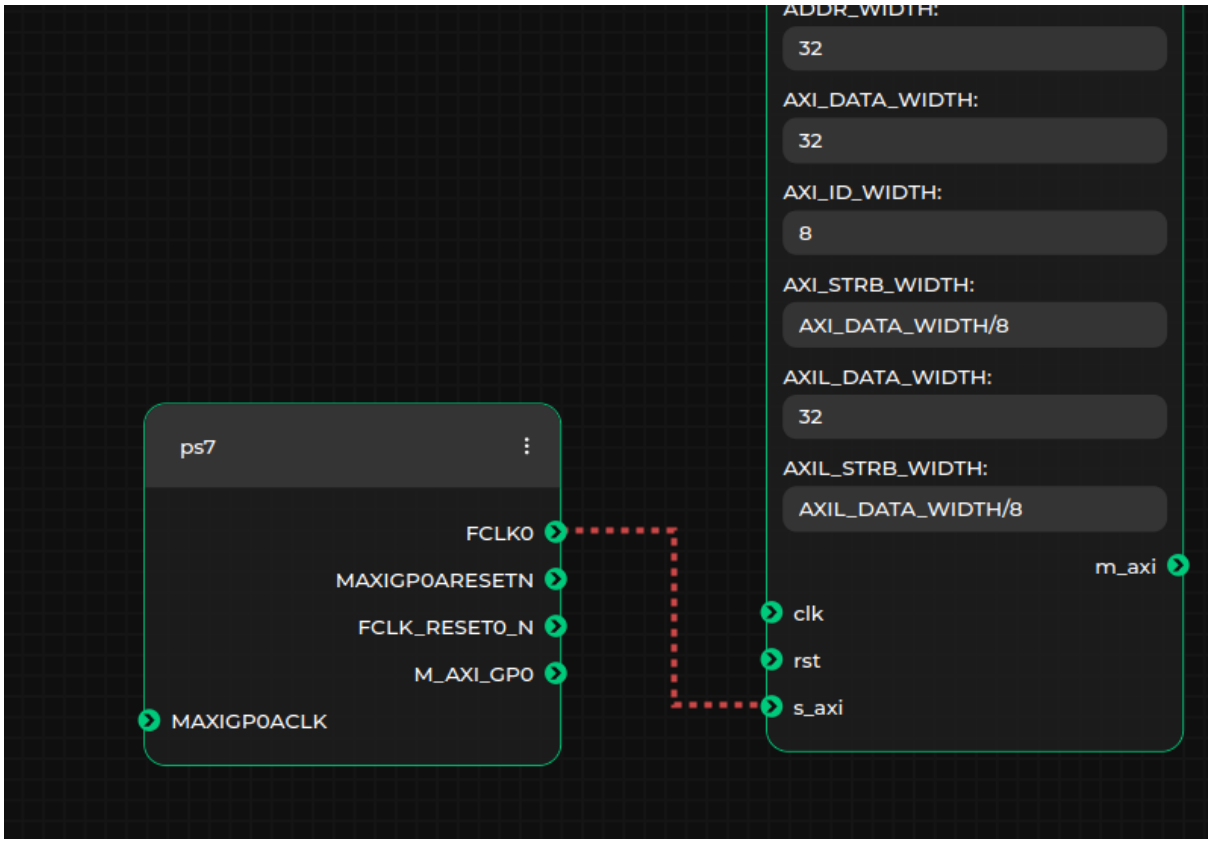
Upon successfully connecting to the server, Topwrap will generate and transmit a specification describing the structure of the selected IP cores. If the `-d` option is specified, the design will be displayed in the GUI. The following content is based on the `PWM` example located in `examples/pwm`. From this point, you can create or modify designs by:

- adjusting the parameter values of IP cores. Each node includes input fields where you can specify parameter values (default values are automatically assigned when an IP core is added to the block design):

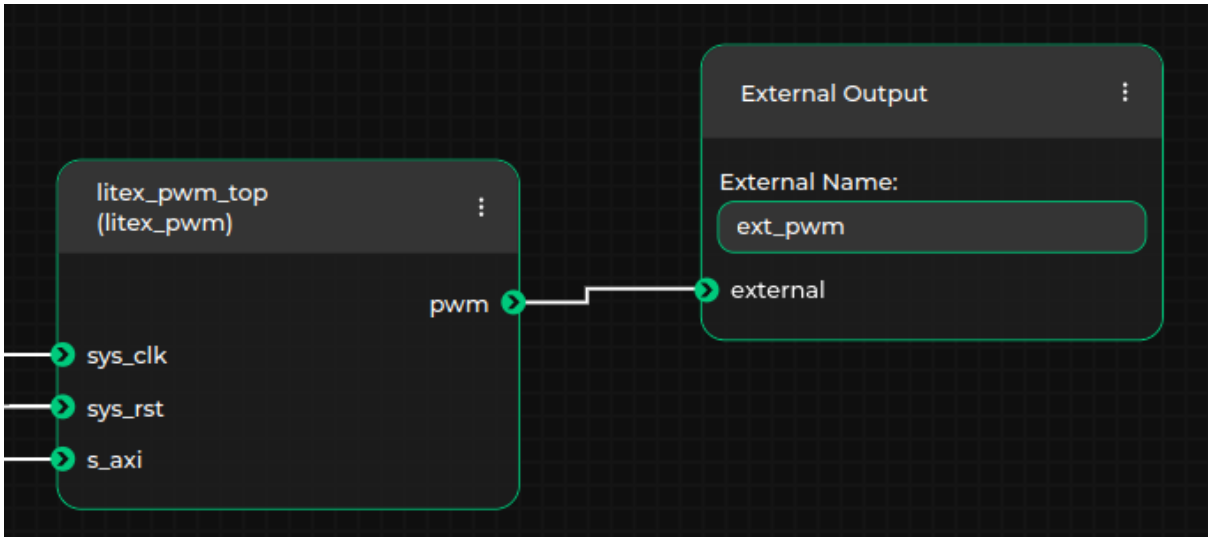


Parameter values can be specified as integers in various bases (e.g., `0x28`, `40`, or `0b101000`) or as arithmetic expressions, which will be evaluated later (e.g., `(AXI_DATA_WIDTH + 1) / 4` is a valid expression, provided a parameter named `AXI_DATA_WIDTH` exists in the same IP core). Additionally, parameter values can be written in Verilog format (e.g., `8'b00011111` or `8'h1F`), in which case they will be interpreted as fixed-width bit vectors

- connecting the ports and interfaces of IP cores. Only connections between ports or interfaces of matching types are allowed. This is automatically validated by the GUI, which uses the type information from the loaded specification. As a result, the GUI will prevent users from making invalid connections (e.g., connecting `AXI4` with `AXI4Lite`, or connecting a port to an interface). A green line will indicate a valid connection, while a red line will indicate an invalid one:



- specifying external ports or interfaces in the top module. To do this, add the appropriate `External Input`, `External Output`, or `External Inout` metanodes, and establish connections between these metanodes and the desired ports or interfaces. Ensure that the name of the external port or interface is updated in the textbox within the selected metanode. For example, in the case where the output pwm port of the `litex_pwm_top` IP core is external to the generated top module, the external port name should be set to `ext_pwm`, as shown below:



An example block design in the Topwrap GUI for the PWM project may look like this:

Note: An interactive KPM frame, where you can explore the block design for this section, is available here in the HTML version of this documentation.

More information about this example can be found here

4.2 Command Line Interface (CLI)

Topwrap has a couple of CLI only functions that expand on the functionality offered by the GUI.

4.2.1 Generating IP core description YAMLs

You can use Topwrap to generate IP core description YAMLs from HDL sources for use in your own `project.yaml`. To learn more about project and core YAMLs, check the *Design description* and *IP description files*.

```
python -m topwrap parse HDL_FILES
```

In HDL source files, ports that belong to the same interface (e.g. `wishbone` or `AXI`) often have a common prefix, which corresponds to the interface name. If the naming convention is followed in the HDL sources, Topwrap can also divide ports into user-specified interfaces, or automatically deduce interface names when generating YAML files:

```
python -m topwrap parse --iface wishbone --iface s_axi HDL_FILES
python -m topwrap parse --iface-deduce HDL_FILES
```

For help, use:

```
python -m topwrap [build|gui|parse] --help
```

CHAPTER

FIVE

SAMPLE PROJECTS

These projects demonstrate how to use Topwrap on a practical level, with examples based on a variety of useful designs.

5.1 Embedded GUI

This section extensively uses an embedded version of Topwrap's GUI to visualize the design of all the examples.

You can use it to explore designs, while adding new blocks, connections, nodes and hierarchies.

The features that require direct connection with Topwrap's backend are not implemented in this demo version, including:

- saving and loading data in `.yaml` files
- building designs
- verifying designs

Tip: Don't forget to use the "Enable fullscreen" button if the viewport is too small.

5.2 Constant

Link to source

> **Note:** An interactive KPM frame, where you can explore the block design for this section, is available here in the HTML version of this documentation.

This example shows how to assign a constant value to a port in an IP core. You can see it in the GUI by using the interactive preview functionality. It is also visible in the description file (`project.yaml`).

Tip: You can find the constant node blueprint in the nodes browser within the `Metanode` section.

5.2.1 Usage

Switch to the subdirectory with the example:

```
cd examples/constant
```

Generate the HDL source:

```
make generate
```

5.3 Inout

Link to source

Note: An interactive KPM frame, where you can explore the block design for this section, is available here in the HTML version of this documentation.

This example showcases the usage of an inout port and its representation in the GUI.

Tip: An inout port is marked in the GUI by a green circle without a directional arrow inside.

The design consists of 3 modules: input buffer `ibuf`, output buffer `obuf`, and bidirectional buffer `iobuf`. Their operation can be described as:

- the input buffer is a synchronous D-type flip flop with an asynchronous reset
- the output buffer is a synchronous D-type flip flop with an asynchronous reset and an `output enable`, which sets the output to a high impedance state (Hi-Z)
- the inout buffer instantiates 1 input and 1 output buffer. The input of the `ibuf` and output of the `obuf` are connected with an inout wire (port).

5.3.1 Usage

Switch to the subdirectory with the example:

```
cd examples/inout
```

Install the required dependencies:

```
pip install -r requirements.txt
```

To generate the bitstream for Zynq, use:

```
make
```

To generate only the HDL sources use:

```
make generate
```

User repository

Link to source

> **Note:** An interactive KPM frame, where you can explore the block design for this section, is available here in the HTML version of this documentation.

This example presents the structure of a user repository containing prepackaged IP cores with sources and custom interface definitions.

Elements of the `repo` directory can be easily reused in different designs by linking to them from the config file or in the CLI.

See also:

For more information about user repositories see *this chapter*.

Tip: As other components of the design are automatically imported from the repository, it's possible to load the entire example by specifying the design file:

```
topwrap gui -d project.yml
```

5.3.2 Usage

Navigate to the `/examples/user_repository/` directory and run:

```
topwrap gui -d project.yml
```

Expected result

Topwrap will load two cores from the `cores` directory, using the interface from the `interfaces` directory.

In the Nodes browser under `IPcore`, two loaded cores: `core1` and `core2`, should be visible.

5.4 Hierarchy

Link to source

> **Note:** An interactive KPM frame, where you can explore the block design for this section, is available here in the HTML version of this documentation.

This example shows how to create a hierarchical design in Topwrap, including a hierarchy that contains IP cores as well as other nested hierarchies.

Check out `project.yaml` to learn how the above design translates to a *design description file*

See also:

For more information, see the section on *Hierarchies*.

Tip: Hierarchies are represented in the GUI by nodes with a green header. To display inner designs, click the `Edit subgraph` option from the context menu.

To exit from the hierarchy subgraph, use the back arrow button on the top left. To add a new hierarchy node, use the `New Graph Node` option in the node browser.

5.4.1 Usage

This example contains the user repository (`repo` directory) and a configuration file for Topwrap (`topwrap.yaml`). It can be loaded by entering the examples directory, and running:

```
topwrap gui -d project.yaml
```

5.5 PWM

Link to source

> **Note:** An interactive KPM frame, where you can explore the block design for this section, is available here in the HTML version of this documentation.

Tip: The IP core in the center of the design (`axi_axil_adapter`) showcases how IP cores with overridable parameters are represented in the GUI.

This is an example of an AXI-mapped PWM IP core that can be generated with LiteX, connected to the ZYNQ Processing System. The core uses the AXILite interface, so a `AXI -> AXILite` converter is needed. You can access its registers starting from address `0x4000000` (the base address of `AXI_GP0` on ZYNQ). The generated signal can be used in a FPGA or connected to a physical port on a board.

Note: To connect I/O signals to specific FPGA pins, you must use mappings in a constraints file. See `zynq.xdc` used in the setup and modify it accordingly.

5.5.1 Usage

Switch to the subdirectory with the example:

```
cd examples/pwm
```

Install the required dependencies:

```
pip install -r requirements.txt
```

Note: In order to generate a bitstream, install Vivado and add it to the `PATH`.

To generate bitstream for Zynq, use:

```
make
```

To generate HDL sources without running Vivado, use:

```
make generate
```

5.6 HDMI

Link to source

Note: An interactive KPM frame, where you can explore the block design for this section, is available here in the HTML version of this documentation.

This is an example of how to use Topwrap to build a complex and synthesizable design.

5.6.1 Usage

Switch to the subdirectory with the example:

```
cd examples/hdmi
```

Install the required dependencies:

```
pip install -r requirements.txt
```

Note: In order to generate a bitstream, install Vivado and add it to the `PATH`.

Generate bitstream for desired target

Snickerdoodle Black:

```
make snickerdoodle
```

Zynq Video Board:

```
make zvb
```

To generate HDL sources without running Vivado, use:

```
make generate
```

5.7 SoC

Link to source

> **Note:** An interactive KPM frame, where you can explore the block design for this section, is available here in the HTML version of this documentation.

This is an example of how to use Topwrap to build a synthesizable SoC design. The SoC contains a VexRiscv core, data and instruction memory, UART and an interconnect that ties all the components together.

5.7.1 Usage

Switch to the subdirectory with the example:

```
cd examples/soc
```

Install the required dependencies:

```
sudo apt install git make g++ ninja-build gcc-riscv64-unknown-elf bsdextrautils
```

Note: To run the simulation, you need:

- verilator

To create and load the bitstream, use:

- Vivado (preferably version 2020.2).
- openFPGALoader (branch)

Generate HDL sources:

```
make generate
```

Build and run the simulation:

```
make sim
```

The expected waveform generated by the simulation is shown in `expected-waveform.svg`.

Generate the bitstream:

```
make bitstream
```

CHAPTER

SIX

CREATING A DESIGN

This chapter explains how to create a design in Topwrap, including a detailed overview of how Topwrap design files are structured.

6.1 Design description

To create a complete and fully synthesizable design, a design file is needed. It is used for:

- specifying interconnects and IP cores
- setting parameter values and describing hierarchies for the project
- connecting the IPs and hierarchies
- picking external ports (those which will be connected to the physical I/O).

You can see example design files in the `examples` directory. The structure of the design file is shown below:

```
ips:
  # specify relations between IP instance names in the
  # design yaml and IP cores description YAMLs
  {ip_instance_name}:
    file: {path_to_ip_description}
  ...

design:
  name: {design_name} # optional name of the toplevel
  hierarchies:
      # see "Hierarchies" below for a detailed description of the format
      ...
  parameters: # specify IP parameter values to be overridden
    {ip_instance_name}:
      {parameters_name} : {parameters_value}
      ...
  ports:
    # specify the incoming ports connections of an IP named `ip1_name`
    {ip1_name}:
      {port1_name} : [{ip2_name}, {port2_name}]
      ...
    # specify the incoming ports connections of a hierarchy named `hier_name`
```

(continues on next page)

(continued from previous page)

```
    {hier_name}:
      {port1_name} : [{ip_name}, {port2_name}]
      ...
    # specify the external port connections
    {ip_instance_name}:
      {port_name} : ext_port_name
    ...

  interfaces:
    # specify the incoming interface connections of the `ip1_name` IP
    {ip1_name}:
      {interface1_name} : [{ip2_name}, {interface2_name}]
      ...
    # specify the incoming interface connections of the `hier_name` hierarchy
    {hier_name}:
      {interface1_name} : [{ip_name}, {interface2_name}]
      ...
    # specify the external interface connections
    {ip_instance_name}:
      {interface_name} : ext_interface_name
    ...

  interconnects:
    # see the "Interconnect generation" page for a detailed description of the␣
↪format
    ...
external: # specify the names of external ports and interfaces of the top module
  ports:
    out:
      - {ext_port_name}
    inout:
      - [{ip_name/hierarchy_name, port_name}]
  interfaces:
    in:
      - {ext_interface_name}
    # note that `inout:` is invalid in the interfaces section
```

`inout` ports are handled differently than the `in` and `out` ports. When an IP has an inout port or when a hierarchy has an inout port specified in its `external.ports.inout` section, it must be included in the `external.ports.inout` section of the parent design. It is required to specify the name of the IP/hierarchy and the port name that contains it. The name of the external port is identical to the one in the IP core. In case of duplicate names, a suffix `$n` is added (where `n` is a natural number) to the name of the second and subsequent duplicate names. `inout` ports cannot be connected to each other.

The design description YAML format allows for creating hierarchical designs. In order to create a hierarchy, add its name as a key in the `design` section and describe the hierarchy design "recursively" by using the same keys and values (`ports`, `parameters` etc.) as in the top-level design (see above). Hierarchies can be nested recursively, which means that you can create a hierarchy inside another one.

Note that IPs and hierarchies names cannot be duplicated on the same hierarchy level. For example, the `design` section cannot contain two identical keys, but it is possible to have `ip_name` key in this section and `ip_name` in the `design` section of a separate hierarchy.

6.1.1 Hierarchies

Hierarchies allow for creating designs with subgraphs in them. The subgraphs can contain multiple IP cores and other subgraphs, allowing for the creation of nested designs in Topwrap.

6.1.2 Format

Hierarchies are specified in the *design description*. The `hierarchies` key must be a direct descendant of the `design` key.

The format is as follows:

```
hierarchies:
    {hierarchy_name_1}:
      ips: # ips that are used on this hierarchy level
        {ip_name}:
          ...

      design:
        parameters:
          ...
        ports: # ports connections internal to this hierarchy.
          # note that also you have to connect port to it's external port␣
↪equivalent (if exists).
          {ip1_name}:
              {port1_name} : [{ip2_name}, {port2_name}]
              {port2_name} : {port2_external_equivalent} # connection to external␣
↪port equivalent. Note that it has to be the parent port.
            ...
        hierarchies:
          {nested_hierarchy_name}:
            # structure here will be the same as for {hierarchy_name_1}
            ...
      external:
        # external ports and/or interfaces of this hierarchy; these can be
        # referenced in the upper-level `ports`, `interfaces` or `external`␣
↪section
        ports:
            in:
              - {port2_external_equivalent}
        ...
    {hierarchy_name_2}:
      ...
```

A more complex example of a hierarchy can be found in the examples/hierarchy directory.

6.2 IP description files

Every IP wrapped by Topwrap needs a description file in the YAML format.

The ports of an IP should be placed in the global `signals` key, followed by the direction - `in`, `out` or `inout`. The module name of an IP should be placed in the global `name` key, and it should be consistent with the definition in the HDL file.

As an example, this is the description of ports in the `Clock Crossing` IP:

```yaml
# file: clock_crossing.yaml
name: cdc_flag
signals:
    in:
        - clkA
        - A
        - clkB
    out:
        - B
```

The previous example can be used with any IP. However, in order to benefit from connecting entire interfaces simultaneously, the ports must belong to a named interface as in this example:

```yaml
#file: axis_width_converter.yaml
name: axis_width_converter
interfaces:
    s_axis:
        type: AXIStream
        mode: subordinate
        signals:
            in:
                TDATA: [s_axis_tdata, 63, 0]
                TKEEP: [s_axis_tkeep, 7, 0]
                TVALID: s_axis_tvalid
                TLAST: s_axis_tlast
                TID: [s_axis_tid, 7, 0]
                TDEST: [s_axis_tdest, 7, 0]
                TUSER: s_axis_tuser
            out:
                TREADY: s_axis_tready
    m_axis:
        type: AXIStream
        mode: manager
        signals:
            in:
                TREADY: m_axis_tready
            out:
                TDATA: [m_axis_tdata, 31, 0]
                TKEEP: [m_axis_tkeep, 3, 0]
                TVALID: m_axis_tvalid
                TLAST: m_axis_tlast
                TID: [m_axis_tid, 7, 0]
```

(continues on next page)

(continued from previous page)

```
                TDEST: [m_axis_tdest, 7, 0]
                TUSER: m_axis_tuser
signals: # These ports do not belong to an interface
    in:
        - clk
        - rst
```

The names `s_axis` and `m_axis` will be used to group the selected ports. Each signal in an interface has a name which must match with the signal that it is connected to, for example `TDATA: port_name` must connect to `TDATA: other_port_name`.

To speed up the generation of YAMLs, Topwrap's `parse` command (see *Generating IP core description YAMLs*) can be used to generate YAMLs from HDL source files.

6.2.1 Port widths

You can specify the port width in the following format:

```
signals:
    in:
        - [port_name, upper_limit, lower_limit]
```

- `port_name` - name of the port.
- `upper_limit` and `lower_limit` define the bit range, where `[upper_limit, lower_limit]` determines the number of bits for the port (e.g. `[63, 0]` for 64 bits).

As an example:

```
signals:
    in:
        - [gpio_io_i, 31, 0] # 32 bits
```

If the bit range is omitted, as in the example below, then the default width of `port_name` is 1 bit.

```
signals:
    in:
      - port_name
```

You can also specify the signal width within interfaces.

```
interfaces:
    s_axis:
        type: AXIStream
        mode: subordinate
        signals:
            in:
                TDATA: [s_axis_tdata, 63, 0] # 64 bits
                ...
                TVALID: s_axis_tvalid # defaults to 1 bit
```

- `TDATA` is assigned to `s_axis_tdata` and is 64 bits wide, defined by `[63, 0]`.

- `TVALID` is assigned to `s_axis_tvalid` and, without a specified range, defaults to `1 bit`.

6.2.2 Parameterization

Port widths don't have to be hardcoded, as parameters can describe an IP core in a generic way, and values specified in IP core YAMLs can be overridden in a design description file (see *Design description*).

```
parameters:
    DATA_WIDTH: 8
    KEEP_WIDTH: (DATA_WIDTH+7)/8
    ID_WIDTH: 8
    DEST_WIDTH: 8
    USER_WIDTH: 1

interfaces:
    s_axis:
        type: AXI4Stream
        mode: subordinate
        signals:
            in:
                TDATA: [s_axis_tdata, DATA_WIDTH-1, 0]
                TKEEP: [s_axis_tkeep, KEEP_WIDTH-1, 0]
                ...
                TID: [s_axis_tid, ID_WIDTH-1, 0]
                TDEST: [s_axis_tdest, DEST_WIDTH-1, 0]
                TUSER: [s_axis_tuser, USER_WIDTH-1, 0]
```

The parameter values can be integers or math expressions.

6.2.3 Port slicing

Ports can be sliced for using some parts of the port as a signal that belongs to a defined interface.

As an example: Port `m_axi_bid` of the IP core is 36 bits wide. Use bits `23..12` as the `BID` signal of the `m_axi_1` AXI manager

```
m_axi_1:
    type: AXI
    mode: manager
    signals:
        in:
            BID: [m_axi_bid, 35, 0, 23, 12]
```

6.3 Interface description files

Topwrap can use predefined interfaces, as illustrated in YAML files that come packaged with the tool. The currently supported interfaces are AXI3, AXI4, AXI Lite, AXI Stream and Wishbone.

An example file looks as follows:

```yaml
name: AXI4Stream
port_prefix: AXIS
signals:
    # The convention assumes the AXI Stream transmitter (manager) perspective
    required:
        out:
            TVALID: tvalid
            TDATA: tdata
            TLAST: tlast
        in:
            TREADY: tready
    optional:
        out:
            TID: tid
            TDEST: tdest
            TKEEP: tkeep
            TSTRB: tstrb
            TUSER: tuser
            TWAKEUP: twakeup
```

The `name` of an interface must be unique.

Signals are either required or optional, and their direction is described from the perspective of the manager (i.e. the direction of signals in the subordinate are flipped). Note that `clock` and `reset` are not included as these are usually inputs to both the manager and subordinate, so they are not supported in the interface specification. Every signal is a key-value pair, where the key is a generic signal name (normally taken from the interface specification) and used to identify it in other parts of Topwrap (i.e. IP core description files), and the value is a regex used to deduce which port defined in the HDL sources represents this signal.

6.3.1 Interface deduction

During *IP core parsing*, you can use the `--iface-deduce` flag to enable automatic pairing of raw ports from HDL sources to interface signals.

This feature matches signal regexes from all available interface descriptions with raw port names of the IP core in order to discover possible interface instances. The pairing is performed on port names that are transformed to lowercase and have the common `port_prefix` removed, which means that the regexes must also be written in lowercase.

6.3.2 Interface compliance

During the *build process*, an optional verification of whether the interface instances used in IP cores are compliant with their respective descriptions can be enabled. The verification consists of checking in the instance if:

- all signals designated as required in the description are included.
- no additional signals beyond those defined in the description are included.

This feature is controlled by the `--iface-compliance` CLI flag or the `force_interface_compliance` key in the *configuration file* and is turned off by default.

CHAPTER

SEVEN

CONFIGURATION

7.1 Configuration file location

The configuration file must be located in one of the following locations:

```
topwrap.yaml
~/.config/topwrap/topwrap.yaml
~/.config/topwrap/config.yaml
```

7.2 Configuration precedence

When multiple configuration files are present, the options are evaluated and overridden based on the location of the configuration file. The precedence, from highest to lowest, is as follows:

- `topwrap.yaml` in the current working directory
- `~/.config/topwrap/topwrap.yaml` (user-specific configuration)
- `~/.config/topwrap/config.yaml` (fallback configuration)

For example, if `force_interface_compliance` is set to `true` in `~/.config/topwrap/config.yaml` but overridden to `false` in `topwrap.yaml`, the latter value will take precedence when running Topwrap in the directory containing `topwrap.yaml`.

7.2.1 Merging strategies for configuration options

Different configuration options use different merging strategies when multiple configuration files are combined:

- **Override** (e.g. `force_interface_compliance`): The value from the higher-precedence file completely replaces the value in lower-precedence files.
- **Merge** (e.g. `repositories`): Values from all configuration files are merged. For example, repositories defined in `~/.config/topwrap/topwrap.yaml` are combined with repositories defined in `topwrap.yaml`.

7.3 Available config options

The configuration file for Topwrap provides the following options:

- `force_interface_compliance`
  - Type: Boolean
  - Default: `false`
  - Merging strategy: Override

  This option enforces compliance with interface definitions when parsing HDLs.

  For more details, refer to *Interface compliance*.

- `repositories`
  - Type: List of objects
  - Merging strategy: Merge
  - Specifies repositories to load, with each repository defined as an object containing the following fields:
    * `name`: (required) The name of the repository.
    * `path`: (required) The file system path to the repository.
  - Example of specifying multiple repositories:

```yaml
repositories:
  - name: name_of_repo
    path: path_to_repo
  - name: another_repo
    path: /absolute/path/to/repo
```

  Repositories are used to package and load multiple IP cores and custom interfaces.

  For more information, refer to *User repositories*.

7.3.1 Example configuration file

Here is a sample configuration file used in the *hierarchy example*

```yaml
force_interface_compliance: true
repositories:
  - name: Hierarchies example
    path: ./repo/
```

CHAPTER

EIGHT

CONSTRUCTING, CONFIGURING AND LOADING REPOSITORIES

By using Topwrap repositories, you can package and load multiple IP cores along with custom interfaces. You can specify the repositories to be loaded each time Topwrap runs by listing them in a *configuration file*.

Topwrap provides an internal API for constructing repositories in Python.

The structure of the repository is as follows:

```
path_to_repository/
|——cores
|   |——ipcore1
|   |   |——srcs
|   |   |   |   file1.v
|   |   |   file1.yaml
|   |
|   |——ipcore2
|       |——srcs
|       |   |   file2.v
|       |   file2.yaml
|
|——interfaces(Optional)
    |   iface1.yaml
    |   iface2.yaml
```

Each repository has two main directories: `cores` and `interfaces`. Inside `cores`, each core has its own directory with a description file and the subdirectory `srcs` where Verilog/VHDL files are stored. The `interfaces` directory is optional, and contains *interface description files*.

A sample user repository can be found in examples/user_repository.

8.1 Using the open source IP cores library with Topwrap

Topwrap comes with built-in support for an extensive library of open source IP cores available through the FuseSoC package manager, which also serves as a build system. This library offers a wide range of reusable IP cores for various applications, enabling easy integration into Topwrap projects. Topwrap simplifies the process of accessing, downloading, and packaging these IP cores, making them readily available for local use in your designs.

To include an IP core from the open source library, there are two methods:

1. **Select the Desired Core**: Browse the available cores (cores_export artifact).

2. **Download and build all available cores**: Use Topwrap's package management command:

```
nox -s package_cores
```

This will download and parse all the cores from Fusesoc into `build/fusesoc_workspace/build/export/cores/`, making them accessible from within Topwrap.

You can learn more about Topwrap integration with FuseSoC *here*

CHAPTER

NINE

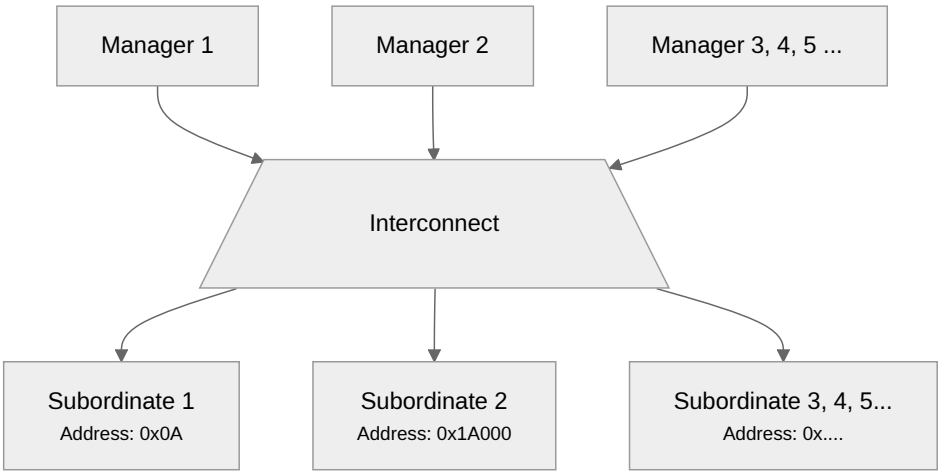
INTERCONNECT GENERATION

Interconnects enable the connection of multiple interfaces in a many-to-many topology, as opposed to the traditional one-to-one manager-subordinate connection. This approach facilitates data transmission between multiple IP cores over a single interface, with the interconnect serving as an middle-man.

> **Warning:** Interconnect generation is an experimental feature.
>
> Currently, creating and showing them is not possible in the Topwrap GUI.

Each manager can communicate with any subordinate connected to the interconnect. Every connected subordinate must be assigned a predefined address range, allowing the interconnect to route data based on the address specified by the manager.

A typical interconnect topology diagram is shown below.



In order to generate an interconnect, you have to describe its configuration in the *Design description* under the `interconnects` key in the following format, as specified below:

9.1 Format

The `interconnects` key must be a direct descendant of the `design` key in the *Design description*.

```
interconnects:
  {interconnect1_name}:
    # Specify clock and reset to drive the interconnect with
    clock: [{ip_name, clk_port_name}]
    reset: [{ip_name, rst_port_name}]
    # Alternatively you can specify a connection to an external port of this␣
↪design:
    # clock: ext_clk_port_name
    # reset: ext_rst_port_name

    # Specify the interconnect type.
    # See the "Supported interconnect types" section below for available types
    # and their characteristics
    type: {interconnect_type}

    # custom parameter values for the specific interconnect type
    parameters:
      {parameters_name1}: parameters_value1
      ...

    # specify managers and their interfaces connected to the bus
    managers:
      {manager1_name}:
        - {manager1_interface1_name}
        ...
      ...

    # specify subordinates, their interfaces connected to the bus and their bus␣
↪parameters
    subordinates:
      {subordinate1_name}:
        {subordinate1_interface1_name}:
          # requests in address range [address, address+size) will be routed to␣
↪this interface
          address: {start_address}
          size: {range_size}
        ...
      ...
  ...
```

9.2 Supported interconnect types

9.2.1 `wishbone_roundrobin`

This interconnect only supports Wishbone interfaces for managers and subordinates. It supports multiple managers, but only one of them can drive the bus at a time (i.e. only one transaction can be happening on the bus at any given moment). A round-robin arbiter decides which manager can currently drive the bus.

Parameters

- `addr_width` - bit width of the address line (addresses access `data_width`-sized chunks)
- `data_width` - bit width of the data line
- `granularity` - access granularity - the smallest unit of data transfer that the interconnect can transfer. Must be: 8, 16, 32, 64
- `features` - optional, list of optional wishbone signals, can contain: `err`, `rty`, `stall`, `lock`, `cti`, `bte`

Known limitations

The currently known limitations are:

- only word-sized addressing is supported (in other words - consecutive addresses can only access word-sized chunks of data)
- crossing clock domains, down-converting (initiating multiple transactions on a narrow bus per one transaction on a wider bus) and up-converting are not supported

CHAPTER

TEN

USING FUSESOC FOR AUTOMATION

Topwrap uses the FuseSoC package manager and build tools for HDL code to automate project generation and the build process. When `topwrap build` is used with the `--fuse` option, it generates a FuseSoC `.core` file along with the top-level wrapper.

10.1 Default tool for synthesis, bitstream generation and programming the FPGA

Topwrap assumes that you're using Vivado. You can change the default tool to something other than Vivado by modifying the generated `.core` file.

10.2 Additional build options

To enable `.core` file generation, supply the `--fuse/-f` flag to `Topwrap build`:

```
topwrap build -d design.yaml --fuse
```

If you have any additional directories with HDL sources or constraint files required for synthesis, you can specify them using the `--sources/-s` option. Sources from these directories get appended to the `filesets.rtl.files` entry in the generated FuseSoC `.core` file.

```
topwrap build -d design.yaml -f --sources ./srcs_v -s ./srcs_vhd
```

If you're targeting a specific FPGA chip, you can additionally specify its number using the `--part/-p` option.

The supplied part number is passed to the FuseSoC `.core` file. It is included in the `targets.default.tools.vivado.part` entry, which is then supplied to Vivado when you run FuseSoC and use the default target. This can be any part number available to your local Vivado installation.

```
topwrap build -d design.yaml -f --part 'xc7z020clg400-3'
```

10.3 .core file template

A template for the `.core` file is bundled with Topwrap (`templates/core.yaml.j2`).

By default, `topwrap.fuse_helper.FuseSocBuilder` searches for the template file in working directory, meaning you must copy the template file into the project location. You may also need to edit the file to change the backend tool, add more targets, set additional `Hooks` and edit other parameters.

10.4 Synthesis

After generating the `.core` file, you can run FuseSoC to generate the bitstream and program the FPGA:

```
fusesoc --cores-root build run {design_name}
```

This requires having a suitable backend tool that is specified in the `.core` file under `targets.default.tools` available in your `PATH` (e.g. Vivado).

CHAPTER

ELEVEN

SETUP

It is required for developers to keep the current code style and it is recommended to frequently run tests.

In order to set up the development environment, install all the optional dependency groups as specified in `pyproject.toml`, which also includes `nox` and `pre-commit`:

```
python -m venv venv
source venv/bin/activate
pip install -e ".[all]"
```

The `-e` option is for installing in editable mode - meaning changes in the code under development will be immediately visible when using the package.

CHAPTER

TWELVE

CODE STYLE

`Nox` or `pre-commit` performs automatic formatting and linting of the code.

12.1 Lint with nox

After successful setup, `Nox` sessions can be executed to perform lint checks:

```
nox -s lint
```

This runs `isort`, `black`, `flake8` and `codespell` and fixes almost all formatting and linting problems automatically, but a small number must be fixed by hand (e.g. unused imports).

Note: To reuse the current virtual environment and avoid lengthy installation processes, use the `-R` flag:

```
nox -R -s lint
```

Note: pre-commit can also be run from nox:

```
nox -s pre_commit
```

12.2 Lint with pre-commit

Alternatively, use `pre-commit` to perform the same job. `pre-commit` hooks need to be installed:

```
pre-commit install
```

Now, each use of `git commit` in the shell will trigger actions defined in the `.pre-commit-config.yaml` file. `pre-commit` is easily deactivated with a similar command:

```
pre-commit uninstall
```

If you wish to run `pre-commit` asynchronously, use:

```
pre-commit run --all-files
```

Note: `pre-commit` by default also runs `nox` with `isort`, `flake8`, `black` and `codespell` sessions.

12.3 Tools

Tools used in the Topwrap project for maintaining the code style:

- `nox` is a tool, which simplifies management of Python testing.
- `pre-commit` is a framework for managing and maintaining multi-language pre-commit hooks.
- `black` is a Python code formatter.
- `flake8` is a tool capable of linting, styling fixes and complexity analysis of Python code.
- `isort` is a Python utility to sort imports alphabetically.
- `codespell` is a Python tool to fix common spelling mistakes in text files

CHAPTER

THIRTEEN

TESTS

Topwrap functionality is validated with tests that leverage the `pytest` library.

13.1 Test execution

The tests are located in the `tests` directory. All tests can be run with `nox` by specifying the `tests` session:

```
nox -s tests
```

This runs tests on the Python interpreter versions that are available locally. There is also a session `tests_in_env` that will automatically install all required Python versions, provided you have pyenv installed:

```
nox -s tests_in_env
```

Note: To reuse an existing virtual environment and avoid lengthy installation times, use the `-R` flag:

```
nox -R -s tests_in_env
```

To force a specific Python version and avoid running tests for all listed versions, use `-p VERSION` flag:

```
nox -p 3.10 -s tests_in_env
```

Tests can also be launched without `nox` by executing:

```
python -m pytest
```

Warning: When running tests by invoking `pytest` directly, tests are ran only on the locally selected Python interpreter. As the CI runs on all supported Python versions, it's recommended to run tests with `nox` on all versions before pushing.

Ignoring a particular test can be performed with `--ignore=test_path`, e.g:

```
python -m pytest --ignore=tests/tests_build/test_interconnect.py
```

For debugging purposes, Pytest captures all output from the test and displays it when all tests are completed. To see the output immediately, pass the `-s` flag to pytest:

```
python -m pytest -s
```

13.2 Test coverage

Test coverage is automatically generated when running tests with `nox`. When invoking `pytest` directly, it can be generated with the `--cov=topwrap` flag. This will generate a summary of coverage, displayed in the CLI.

```
python -m pytest --cov=topwrap
```

Additionally, the summary can be generated in HTML with the flags `--cov=topwrap --cov-report html`, where lines that were not covered by tests can be browsed:

```
python -m pytest --cov=topwrap --cov-report html
```

The generated report is available at `htmlcov/index.html`

13.3 Updating kpm test data

All `kpm` data from examples can be generated using `nox`. This is useful when changing Topwrap functionality relating to kpm, as it avoids manually changing test data in every sample. Users can either update of example data such as the specification or update everything (dataflows, specifications, designs).

To update everything run:

```
nox -s update_test_data
```

To update only specifications run:

```
nox -s update_test_data -- specification
```

Valid options for `update_test_data` sessions, are:

- `specification`
- `dataflow`
- `design`

CHAPTER

FOURTEEN

WRAPPER

`Wrapper` is an abstraction over entities that have ports. Examples include IP cores written in Verilog/VHDL, cores written in Amaranth and hierarchical collections for these that expose some external ports.

Subclasses of this class have to supply an implementation of the property `get_ports()`, which has to return a list of all ports in the entity.

class Wrapper(**args, src_loc_at=0, **kwargs*)

Base class for modules that want to connect to each other.

Derived classes must implement get_ports method that returns a list of WrapperPort's - external ports of a class that can be used as endpoints for connections.

__init__(*name: str*)

get_port_by_name(*name: str*) → *WrapperPort*

Given port's name, return the port as WrapperPort object.

Raises

ValueError – If such port doesn't exist.

get_ports() → List[*WrapperPort*]

Return a list of external ports.

get_ports_of_interface(*iface_name: str*) → List[*WrapperPort*]

Return a list of ports of specific interface.

Raises

ValueError – if such interface doesn't exist.

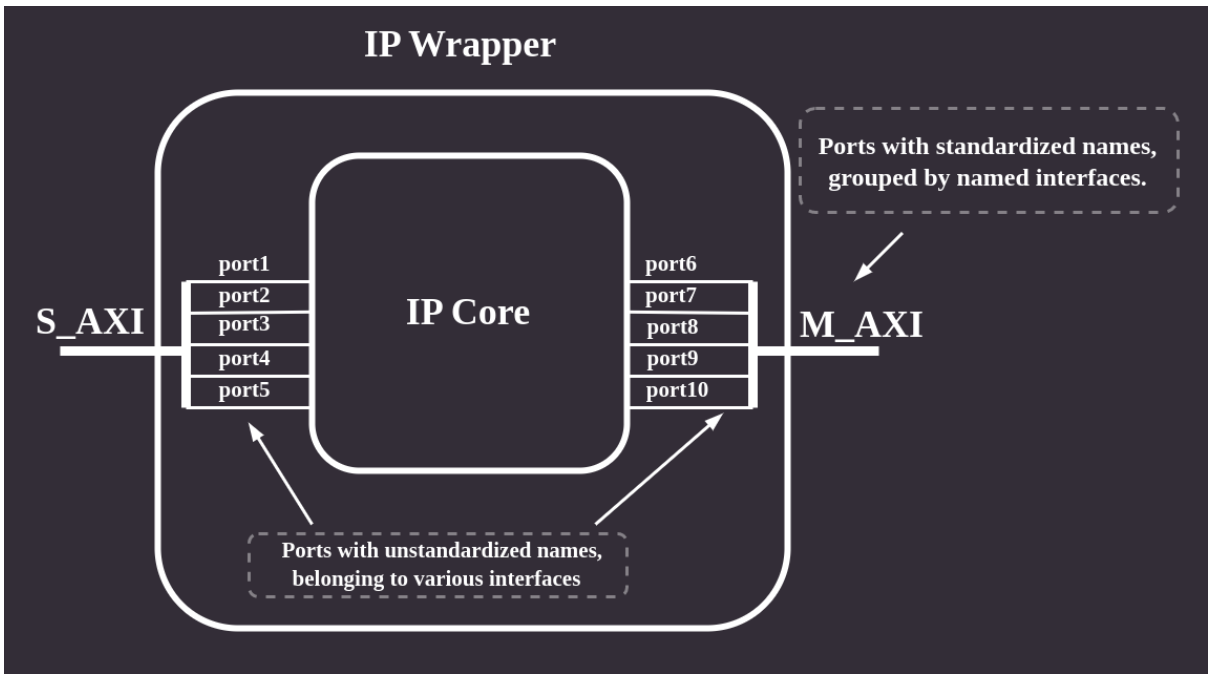
CHAPTER

FIFTEEN

IPWRAPPER CLASS

`IPWrapper` provides an abstraction over a raw HDL source file. Instances of this class can be created from the loaded IP-core YAML description.

It creates an Amaranth `Instance` object during elaboration, referencing a particular HDL module and appears as a module instantiation in the generated top level. Ports and interfaces (lists of ports) can be retrieved via standard methods of `Wrapper`. These are instances of `WrapperPorts`.



class IPWrapper(**args, src_loc_at=0, **kwargs*)

This class instantiates an IP in a wrapper to use its individual ports or grouped ports as interfaces.

__init__(yaml_path: *Path*, ip_name: *str*, instance_name: *str*, params={})

Parameters

yaml_path: Path
path to the IP Core description yaml file

ip_name: str
name of the module to wrap

instance_name: str
name of this instance

params={}
optional, HDL parameters of this instance

`get_ports()` → List[*WrapperPort*]

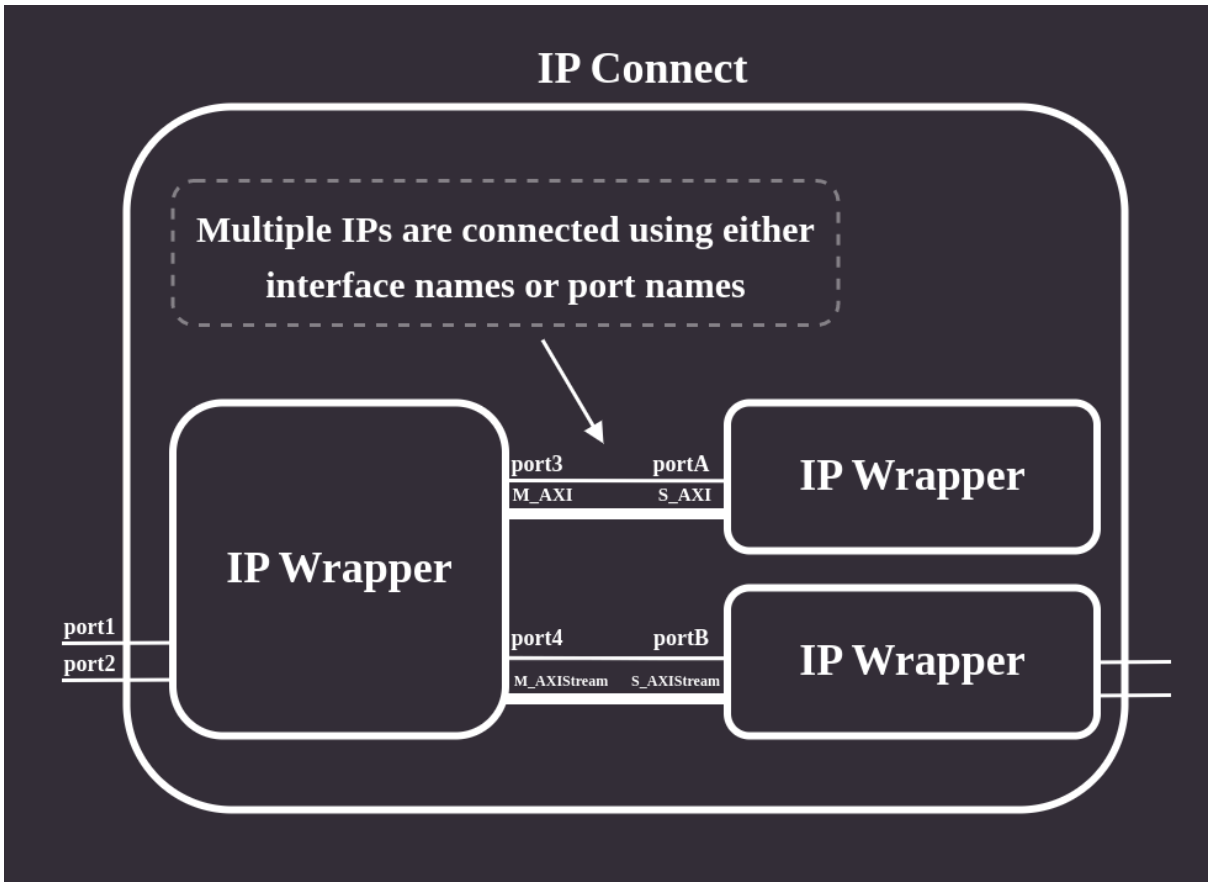
Return a list of all ports that belong to this IP.

CHAPTER

SIXTEEN

IPCONNECT CLASS

IPConnect provides the means of connecting ports and interfaces of objects that are subclasses of Wrapper. Since IPConnect is a subclass of Wrapper itself, this means that it also has IO - ports and interfaces, and that multiple IPConnects can have their ports and interfaces connected to each other (or other objects that subclass Wrapper).



Instances of Wrapper objects can be added to an IPConnect using add_component() method:

```
# create an IP wrapper
dma = IPWrapper('DMATop.yaml', ip_name='DMATop', instance_name='DMATop0')
ipc = IPConnect()
ipc.add_component("dma", dma)
```

Connections between cores can then be made with connect_ports() and connect_interfaces() based on names of the components and names of ports/interfaces:

```
ipc.connect_ports("comp1_port_name", "comp1_name", "comp2_port_name", "comp2_name
↪")
ipc.connect_interfaces("comp1_interface_name", "comp1_name", "comp2_interface_name
↪", "comp2_name")
```

Setting ports or interfaces of a module added to `IPConnect` as external with `_set_port()` and `_set_interface()` and allows these ports/interfaces to be connected to other `Wrapper` instances.

```
ipc._set_port("comp1_name", "comp1_port_name", "external_port_name")
ipc._set_interface("comp1_name", "comp1_interface_name", "external_interface_name
↪")
```

This is done automatically in the `make_connections()` method when the design is built, based on the data from the YAML design description.

class IPConnect(**args, src_loc_at=0, **kwargs*)

Connector for multiple IPs, capable of connecting their interfaces as well as individual ports.

__init__(*name: str = 'ip_connector'*)

_connect_external_ports(internal: WrapperPort, external: WrapperPort)

Makes a pass-through connection - port of an internal module in IPConnect is connected to an external IPConnect port.

Parameters

internal: ***WrapperPort***
port of an internal module of IPConnect

external: ***WrapperPort***
external IPConnect port

_connect_internal_ports(port1: WrapperPort, port2: WrapperPort)

Connects two ports with matching directionality. Disallowed configurations are: - input to input - output to output - inout to inout All other configurations are allowed.

Parameters

port1: ***WrapperPort***
1st port to connect

port2: ***WrapperPort***
2nd port to connect

_connect_to_external_port(internal_port: *str*, internal_comp: *str*, external_port: *str*, external: *DesignExternalPorts*) → None

Connect internal port of a component to an external port

Parameters

internal_port: str
internal port name in internal_component to connect to external_port

internal_comp: str
internal component name

external_port: str
external port name

external: DesignExternalPorts
dictionary in the form of {"in": list, "out": list, "inout": list} containing port names specified as external in each of the three categories. All keys are optional and lack of a category implies an empty list

`_set_interface`(comp_name: *str*, iface_name: *str*, external_iface_name: *str*) → None

Set interface specified by name as an external interface

Parameters

comp_name: str
name of the component - hierarchy or IP core

iface_name: str
interface name in the component

external_iface_name: str
external name of the interface specified in "external" section

Raises
`ValueError` – if such interface doesn't exist

`_set_port`(comp_name: *str*, port_name: *str*, external_name: *str*) → None

Set port specified by name as an external port

Parameters

comp_name: str
name of the component - hierarchy or IP core

port_name: str
port name in the component

external_name: str
external name of the port specified in "external" section

Raises
`ValueError` – if such port doesn't exist

`add_component`(name: *str*, component: Wrapper) → None

Add a new component to this IPConnect, allowing to make connections with it

Parameters

name: str
name of the component

component: *Wrapper*
Wrapper object

`connect_interfaces`(iface1: *str*, comp1_name: *str*, iface2: *str*, comp2_name: *str*) → None

Make connections between all matching ports of the interfaces

Parameters

iface1: str
name of the 1st interface

comp1_name: str
name of the 1st IP

iface2: str
name of the 2nd interface

comp2_name: str
name of the 2nd IP

Raises
`ValueError` – if any of the IPs doesn't exist

`connect_ports`(port1_name: *str*, comp1_name: *str*, port2_name: *str*, comp2_name: *str*) → None

Connect ports of IPs previously added to this Connector

Parameters

port1_name: str
name of the port of the 1st IP

comp1_name: str
name of the 1st IP

port2_name: str
name of the port of the 2nd IP

comp2_name: str
name of the 2nd IP

Raises
`ValueError` – if such IP doesn't exist

`get_ports`() → list

Return a list of external ports of this module

`make_connections`(ports: *DS_PortsT*, interfaces: *DS_InterfacesT*, external: *DesignExternalSection*) → None

Use names of port and names of ips to make connections

Parameters

ports: DS_PortsT
"ports" section in the YAML design specification

interfaces: DS_InterfacesT
"interfaces" section in the YAML design specification

external: DesignExternalSection
"external" section in the YAML design specification

`make_interconnect_connections`(interconnects: *Dict[str, DesignSectionInterconnect]*, external: *DesignExternalSection*)

Connect subordinates and managers to their respective interfaces in the interconnect

Parameters

interconnects: Dict[str, DesignSectionInterconnect]
"interconnects" section in the YAML design specification

external: DesignExternalSection
"external" section in the YAML design specification

`set_constant`(comp_name: *str*, comp_port: *str*, target: *int*) → None

Set a constant value on a port of an IP

Parameters

comp_name: str
name of the IP or hierarchy

comp_port: str
name of the port of the IP or hierarchy

target: int
int value to be assigned

Raises
`ValueError` – if such IP doesn't exist

`validate_inout_connections`(inouts: *Collection[Tuple[str, str]]*)

Checks that all inout ports of any IP or hierarchy in the design are explicitly listed in the 'external' section.

Parameters

inouts: Collection[Tuple[str, str]]
external.ports.inout section of the design description YAML

CHAPTER

SEVENTEEN

ELABORATABLEWRAPPER CLASS

`ElaboratableWrapper` encapsulates an Amaranth's Elaboratable and exposes an interface compatible with other wrappers, allowing for making connections with them.

The supplied elaboratable must contain the `signature` property and a conforming interface as specified by the Amaranth docs.

The names, directionality and widths of ports are inferred from it.

class ElaboratableWrapper(**args*, *src_loc_at=0*, ***kwargs*)

Allows connecting an Amaranth's Elaboratable with other classes derived from Wrapper.

__init__(name: *str*, elaboratable: *Elaboratable*)

Parameters

name: str
name of this wrapper

elaboratable: Elaboratable
Amaranth's Elaboratable object to wrap

get_ports() → List[*WrapperPort*]

Return a list of external ports.

get_ports_hier() → Mapping[str, Signal | Mapping[str, Signal | SignalMapping]]

Maps elaboratable's Signature to a nested dictionary of WrapperPorts. See _gather_signature_ports for more details.

CHAPTER

EIGHTEEN

WRAPPER PORT

The class `WrapperPort` is an extension to Amaranth's `Signal`. It wraps a port, adding a new name and optionally slicing the signal. It adds these attributes:

```
WrapperPort.internal_name     # name of the port in internal source to be wrapped
WrapperPort.direction         # DIR_FANIN, DIR_FANOUT or DIR_NONE
WrapperPort.interface_name    # name of the group of ports (interface)
WrapperPort.bounds            # range of bits that belong to the port
                              # and the range which is sliced from the port
```

See *IP core port-slicing* to know more about `bounds`.

This is used in the `IPWrapper` class implementation and there should be no need to use `WrapperPort` individually.

> **Warning:** `WrapperPort` is scheduled to be replaced in favor of Amaranth's `Signal` so it should not be used in any new functionality.

class WrapperPort(*shape=None, src_loc_at=0, **kwargs*)

__init__(*shape=None, src_loc_at=0, *, bounds: List[int], name: str, internal_name: str, interface_name: str | None = None, direction: PortDirection*)

Wraps a port, adding a new name and optionally slicing the signal

Parameters

bounds: List[int]
4-element list where: [0:1] - upper and lower bounds of reference signal, [2:3] - upper and lower bounds of internal port, which are either the same as reference port, or a slice of the reference port

name: str
a new name for the signal

internal_name: str
name of the port to be wrapped/sliced

interface_name: str | None = None
name of the interface the port belongs to

direction: PortDirection
one of PortDirection, e.g. DIR_OUT

static like(other, **kwargs)

Creates a WrapperPort object with identical parameters as other object

Parameters

other
object to clone data from

****kwargs**
optional constructor parameters to override

CHAPTER

NINETEEN

FUSESOCBUILDER

Topwrap supports generating FuseSoC `.core` files with `FuseSocBuilder`.The `.core` file contains information about source files and synthesis tools.

Generation of FuseSoC `.core` files is based on a Jinja template that defaults to `topwrap/templates/core.yaml.j2`, but can be overridden.

Here's an example of how to generate a simple project:

```
from topwrap.fuse_helper import FuseSocBuilder
fuse = FuseSocBuilder()

# add source of the IPs used in the project
fuse.add_source('DMATop.v', 'verilogSource')

# add source of the top file
fuse.add_source('top.v', 'verilogSource')

# specify the names of the core file and the directory where sources are stored
# generate the project
fuse.build('build/top.core', 'sources')
```

> **Warning:** Default template in `topwrap/templates/core.yaml.j2` does not make use of resources added with `add_dependency()` or `add_external_ip()`, i.e. they won't be present in the generated core file.

class FuseSocBuilder(*part*)

Use this class to generate a FuseSoC .core file

__init__(*part*)

add_dependency(*dependency: str*)

Adds a dependency to the list of dependencies in the core file

add_external_ip(*vlnv: str*, *name: str*)

Store information about IP Cores from Vivado library to generate hooks that will add the IPs in a TCL script.

add_source(*filename*, *type*)

Adds an HDL source to the list of sources in the core file

add_sources_dir(*sources_dir: Collection[Path], core_path: Path*)

Given a name of a directory, add all files found inside it. Recognize VHDL, Verilog, and XDC files.

build(*top_name: str, core_path: Path,* sources_dir*: Collection[Path] = [],* template_name*: str | None = None*)

Generate the final create .core file

Parameters

sources_dir: Collection[Path] = []
additional directory with source files to add

template_name: str | None = None
name of jinja2 template to be used, either in working directory, or bundled with the package. defaults to a bundled template

CHAPTER

TWENTY

INTERFACE DEFINITION

Topwrap uses *interface definition files* for its parsing functionality.

These are used to match a given set of signals that appear in the HDL source with signals in the interface definition.

`InterfaceDefinition` is defined as a `marshmallow_dataclass.dataclass` - this enables loading the YAML structure into Python objects and performs validation (that the YAML is in the correct format) and typechecking (that the loaded values are of the correct types).

class InterfaceDefinition(*name: str, port_prefix: str, signals: ~topwrap.interface.InterfaceDefinitionSignals = <factory>*)

Interface described in YAML interface definition file

__init__(*name: str, port_prefix: str, signals: ~topwrap.interface.InterfaceDefinitionSignals = <factory>*)

Schema

alias of `InterfaceDefinition`

get_interface_by_name(*name: str*) → *InterfaceDefinition* | None

Retrieve interface definition by its name

Returns

InterfaceDefinition object, or *None* if there's no such interface

CHAPTER

TWENTYONE

CONFIG

The `Config` object stores configuration values. The global `topwrap.config.Config` object is used throughout the codebase to access the Topwrap configuration.

It is created by `ConfigManager` that reads the config files as defined in `topwrap.config.ConfigManager.DEFAULT_SEARCH_PATHS`, with local files taking precedence.

class Config(*force_interface_compliance: bool | None = False, repositories: ~typing.List[~topwrap.config.RepositoryEntry] | None = <factory>*)

Global topwrap configuration

__init__(*force_interface_compliance: bool | None = False, repositories: ~typing.List[~topwrap.config.RepositoryEntry] | None = <factory>*)

Schema

alias of `Config`

class ConfigManager(*search_paths: List[PathLike] | None = None*)

Manager used to load topwrap's configuration from files.

The configuration files are loaded in a specific order, which also determines the priority of settings that are defined differently in the files. The list of default search paths is defined in the *DEFAULT_SEARCH_PATH* class variable. Configuration files that are specified earlier in the list have higher priority and can overwrite the settings from the files that follow. The default list of search paths can be changed by passing a different list to the ConfigManager constructor.

__init__(*search_paths: List[PathLike] | None = None*)

CHAPTER

TWENTYTWO

DEDUCING INTERFACES

This section describes how inferring interfaces works when using `topwrap parse` with `--iface-deduce`, `--iface` or `--use-yosys` options.

The problem can be described as: given a set of signals, infer what interfaces are present in this set and assign the signals to the appropriate interfaces. Interface names and types (AXI4, AXI Stream, Wishbone, etc.) are generally not provided in advance. The algorithm implemented in Topwrap works broadly as follows:

1. Split the given signal set into disjoint subsets of signals based on common prefixes in their names
2. For a given subset, try to pair each signal name (as it appears in the RTL) with the name of an interface signal (as it is defined in the specification of a particular interface). This pairing is called "a matching", and matching with signals from all defined interfaces is tried.
3. For a given subset and matched interface, infer the interface direction (manager/subordinate) based on the direction of a signal in this set.
4. Compute the score for each matching, e.g. if signal names contain `cyc`, `stb` and `ack` (and possibly more) it's likely that this set is a Wishbone interface. Among all interfaces, the interface that has the highest matching score is selected.

22.1 Step 1 - splitting ports into subsets

First, all ports of a module are grouped into disjoint subsets. Execution of this step differs based on the options supplied to `topwrap parse`:

- with `--iface` the user supplies Topwrap with interface names - ports with names starting with a given interface name will be put in the same subset.
- with `--use-yosys` grouping is done by parsing the RTL source with `yosys`, where ports have attributes in the form of `(* interface="interface_name" *)`. Ports with the same `interface_name` will be put in the same subset.
- with `--iface-deduce` grouping is done by computing longest common prefixes among all ports. This is done with the help of a trie and only allows prefixes that would split the port name on an underscore (e.g. in `under_score` valid prefixes are an empty string, `under` and `under_score`) or a camel-case word boundary (e.g. in `wordBoundary` valid prefixes are an empty string, `word` and `wordBoundary`). As with user-supplied prefixes, ports with names starting with a given prefix will be put in the same subset.

22.2 Step 2 - matching ports with interface signal names

Given a subset of ports from the previous step, this step tries to match a regexp from an interface definition YAML for a given interface signal to one of the port names and returns a collection of pairs: RTL port + interface port. For example, when matching against AXI4, a port named `axi_a_arvalid` should match to an interface port named `ARVALID` in the interface definition YAML.

This operation is performed for all defined interfaces for a given subset of ports. The overall result of this step is a collection of matchings. For most interfaces these matchings will be poor - e.g. `axi_a_arvalid` or other AXI4 signals won't match to most Wishbone interface signals, but an interface that a human would usually assign to a given set of signals will have most signals matched.

22.3 Step 3 - inferring interface direction

This step picks a representative RTL signal from a single signal matching from the previous step and checks its direction against direction of the corresponding interface signal in the interface definition YAML - if it's the same then it's a manager interface (since the convention in interface description files is to describe signals from the manager's perspective), otherwise it's a subordinate.

22.4 Step 4 - computing interface matching score

This step computes a score for each matching returned by Step 2. The score is based on the number of matched/unmatched optional/required signals in each matching.

Not matching some signals in a given group (from step 1.) is heavily penalized to encourage selecting an interface that "fits" a given group best. For example, AXI Lite is a subset of AXI4, so a set of signals that should be assigned AXI4 interface could very well fit the description of AXI Lite, but this mechanism discourages selecting such matching in favor of selecting the other.

Not matching some signals of a given interface (from interface description YAML) is also penalized. Inverting the previous example, a set of signals that should be assigned AXI Lite interface could very well fit the description of AXI4, but because it's missing a few AXI4 signals, selecting this matching is discouraged in favor of selecting the other.

22.4.1 High scoring function

A well-behaving scoring function should satisfy some properties to ensure that the best "fitting" interface is selected. To describe these we introduce the following terminology:

- >/>=/== should be read as "must have a greater/greater or equal/equal score than".
- Partial matching means matching where some RTL signals haven't been matched to interface signals, full matching means matching where all have been matched.

Current implementation when used with default config values satisfies these properties:

1. full matching with N+1 signals matched (same type) == full matching with N signals matched (same type)

2. full matching with N signals matched (same type) > partial matching with N signals matched (same type)
3. partial matching with N+1 signals matched (same type) > partial matching with N signals matched (same type)
4. full matching with N+1 required, M+1 optional signals >= full matching with N+1 optional, M optional signals >= full matching with N required, M+1 optional signals >= full matching with N required, M optional signals

Properties 2-4 generally ensure that interfaces with more signals matched are favored more than those with less signals matched. Property 1 follows from the current implementation and is not needed in all implementations.

Full details can be found in the implementation itself.

CHAPTER

TWENTYTHREE

EXAMPLES

Note: The basic usage of examples are explained in the *Getting started* section.

The examples provided with this project cover very simple designs through to complex fully synthesizable cores.

They are sorted by increasing levels of complexity and the number of used features, e.g.:

- 101: minimal base design
- 102: introduce user to parameters
- 103: introduce user to slicing
- 104: introduce user to interfaces
- 105: etc.

Developers are encouraged to create/add new examples in the same manner, as simple examples are used to teach how to use this tool and demonstrate its features. Real-world use cases are also welcome to show that the implementation is mature enough to handle practical designs.

CHAPTER

TWENTYFOUR

FUTURE PLANNED ENHANCEMENTS IN TOPWRAP

24.1 Library of open-source cores

Currently, users have to manually or semi-manually (e.g. through FuseSoC) supply all of the cores used in the design. In future, a repository of open-source cores that can be easily reused will be provided, allowing users to quickly put together designs from premade hardware blocks.

24.2 Support for hierarchical block designs in Topwrap's GUI

Topwrap supports creating hierarchical designs by manually writing the hierarchy in the design description YAML, while in future, this feature will be additionally supported in the GUI for visually organizing complex designs.

24.3 Support for parsing SystemVerilog sources

Information about IP cores is stored in *IP description files*. These files can be generated automatically from HDL source files - currently Verilog and VHDL are supported. In a future release, Topwrap will also provide the possibility of generating YAMLs from SystemVerilog.

CHAPTER

TWENTYFIVE

OTHER POSSIBLE IMPROVEMENTS

25.1 Ability to produce top-level wrappers in VHDL

Topwrap currently uses Amaranth to generate top-level designs in Verilog. We would also like to add the ability to produce such designs in VHDL.

25.2 Bus management

Another feature that could be introduced is allowing users to create full-featured designs with processors by providing proper support for bus management.

This should include features such as:

- the ability to specify the address of a peripheral device on the bus
- support for the most popular buses (AXI, TileLink, Wishbone)

This will require writing or creating bus arbiters (`round-robin`, `crossbar`) and providing a mechanism for connecting manager(s) and subordinate(s) together. As a result, the user would be able to create complex SoCs directly in Topwrap.

Currently, only experimental support for `Wishbone with a round-robin arbiter` is available.

25.3 Improve the process of recreating a design from a YAML file

One of the main features supported by Topwrap and the GUI is exporting and importing user-created designs, both to or from a *design description* YAML. However, during these conversions, information about the position of user-added nodes is not preserved. This is cumbersome in the case of complicated designs since the imported nodes are not retained in the most optimal positions.

Therefore, one of our objectives is to provide a convenient way of creating and restoring user-created designs in the GUI, so that the node positions are retained.

25.4 Deeper integration with other tools

Topwrap can build designs, but testing and synthesis rely on the user - they have to automate this process themselves (e.g. with makefiles). To improve the usability of Topwrap, a potential area of improvement is to integrate tools for synthesis, simulation and co-simulation (with e.g. Renode with Topwrap, accessible through scripts. Some could be pre-packaged with Topwrap (e.g. simulation with Verilator, synthesis with Vivado).

It could also be possible to invoke these from the GUI by adding custom buttons or through the integrated terminal.

25.5 Provide a way to parse HDL sources from the GUI level

Another issue related to HDL parsing is that the user has to manually parse HDL sources to obtain the IP core description YAMLs. These files then need to be provided as command-line parameters when launching the Topwrap GUI client application. Therefore, we aim to provide a way of parsing HDL files directly from the GUI.

CHAPTER

TWENTYSIX

USING KPM IFRAMES INSIDE DOCS

It is possible to use the `kpm_iframe` Sphinx directive to embed KPM directly inside a doc.

26.1 Usage

```
```{kpm_iframe}
:spec: <KPM specification .json file URI>
:dataflow: <KPM dataflow .json file URI>
:preview: <a Boolean value specifying whether this KPM should be started in␣
↪preview mode>
:height: <a string CSS height property that sets the `height` of the iframe>
:alt: <a custom alternative text used in the PDF documentation instead of the␣
↪default one>
```
```

`URI` represents either a local file from sources that are copied into the build directory, or a remote resource.

All parameters in this directive are optional.

26.2 Tests

26.2.1 Use remote specification

Note: The graph below is supposed to be empty.

It doesn't load a dataflow, only a specification that provides IP-cores to the Nodes browser on the sidebar.

Note: An interactive KPM frame, where you can explore the block design for this section, is available here in the HTML version of this documentation.

26.2.2 Use local files

Note: An interactive KPM frame, where you can explore the block design for this section, is available here in the HTML version of this documentation.

26.2.3 Open in preview mode

Note: An interactive KPM frame, where you can explore the block design for this section, is available here in the HTML version of this documentation.

26.2.4 Use a custom alt text

Note: The alternative text is visible instead of the iframe in the PDF version of this documentation.

Note: This diagram showcases the block design of the "hierarchy" example

INDEX

Symbols

__init__() (*Config method*), 59
__init__() (*ConfigManager method*), 59
__init__() (*ElaboratableWrapper method*), 53
__init__() (*FuseSocBuilder method*), 56
__init__() (*IPConnect method*), 49
__init__() (*IPWrapper method*), 46
__init__() (*InterfaceDefinition method*), 58
__init__() (*Wrapper method*), 45
__init__() (*WrapperPort method*), 54
_connect_external_ports() (*IPConnect method*), 49
_connect_internal_ports() (*IPConnect method*), 49
_connect_to_external_port() (*IPConnect method*), 49
_set_interface() (*IPConnect method*), 50
_set_port() (*IPConnect method*), 50

A

add_component() (*IPConnect method*), 50
add_dependency() (*FuseSocBuilder method*), 56
add_external_ip() (*FuseSocBuilder method*), 56
add_source() (*FuseSocBuilder method*), 56
add_sources_dir() (*FuseSocBuilder method*), 56

B

build() (*FuseSocBuilder method*), 57

C

Config (*class in topwrap.config*), 59
ConfigManager (*class in topwrap.config*), 59
connect_interfaces() (*IPConnect method*), 50
connect_ports() (*IPConnect method*), 51

E

ElaboratableWrapper (*class in topwrap.elaboratable_wrapper*), 53

F

FuseSocBuilder (*class in topwrap.fuse_helper*), 56

G

get_interface_by_name() (*in module topwrap.interface*), 58
get_port_by_name() (*Wrapper method*), 45
get_ports() (*ElaboratableWrapper method*), 53
get_ports() (*IPConnect method*), 51
get_ports() (*IPWrapper method*), 47
get_ports() (*Wrapper method*), 45
get_ports_hier() (*ElaboratableWrapper method*), 53
get_ports_of_interface() (*Wrapper method*), 45

I

InterfaceDefinition (*class in topwrap.interface*), 58
IPConnect (*class in topwrap.ipconnect*), 49
IPWrapper (*class in topwrap.ipwrapper*), 46

L

like() (*WrapperPort static method*), 54

M

make_connections() (*IPConnect method*), 51
make_interconnect_connections() (*IPConnect method*), 51

S

Schema (*Config attribute*), 59
Schema (*InterfaceDefinition attribute*), 58
set_constant() (*IPConnect method*), 52

V

validate_inout_connections() (*IPConnect method*), 52

W

`Wrapper` (*class in topwrap.wrapper*), 45
`WrapperPort` (*class in topwrap.amaranth_helpers*), 54